

1. AIM

To design and implement a Digital Payment System in Java using object-oriented programming concepts, simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.

2. DESCRIPTION

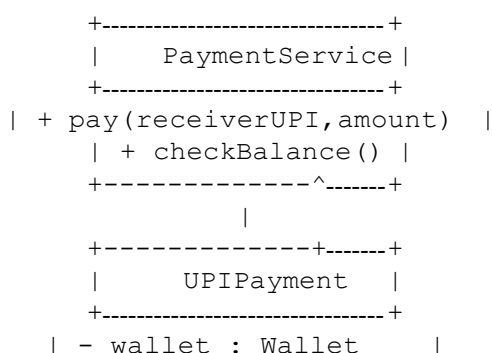
The Digital Payment System is designed for a user with a digital wallet. The system allows the user to create a wallet using a name, mobile number, and UPI ID, add money to the wallet, make payments to another user's UPI ID, check the available balance, and validate transactions before processing them.

The program uses a `PaymentService` interface to define common payment operations. The `Wallet` class stores user information and balance using private data members to demonstrate encapsulation. The `UPIPayment` class implements the `PaymentService` interface and performs UPI payments. Custom exceptions handle invalid UPI IDs, invalid transaction amounts, and insufficient balance.

3. ALGORITHM

1. Start the program.
2. Create `PaymentService` and declare `pay()` and `checkBalance()`.
3. Create `Wallet` with private `userName`, `mobileNumber`, `upild`, and `balance`.
4. Create a constructor and initialize wallet details and balance.
5. Define `addMoney()`, `getBalance()`, and `displayWalletDetails()`.
6. Create custom exceptions for invalid UPI, invalid amount, and insufficient balance.
7. Create `UPIPayment` and implement `PaymentService`.
8. Associate a `Wallet` object with `UPIPayment`.
9. Validate the receiver UPI ID using String operations.
10. Throw `InvalidUPIException` if the UPI ID is invalid.
11. Throw `InvalidAmountException` if the amount is zero or negative.
12. Throw `InsufficientBalanceException` if the balance is insufficient.
13. Deduct the amount when all validations succeed.
14. Display the payment success message.
15. Use try-catch-finally for exception handling.
16. Display the final wallet details and balance.
17. Stop the program.

4. UML DIAGRAM



```

+-----+
| + pay() |
| + checkBalance() |
+-----+
|
+-----v-----+
|      Wallet      |
+-----+
| - userName      |
| - mobileNumber  |
|| - balance      |
+-----+
| + addMoney()    |
| + getBalance()  |
| + deductMoney() |
| + displayWalletDetails() |
+-----+

```

```

- upiId

```

5. SOURCE CODE

// Interface

```
interface PaymentService {  
    void pay(String upild, double amount);  
    void checkBalance();  
}
```

// Custom Exception 1

```
class InsufficientBalanceException extends Exception {  
    InsufficientBalanceException(String message) {  
        super(message);  
    }  
}
```

// Custom Exception 2

```
class InvalidUPIException extends Exception {  
    InvalidUPIException(String message) {  
        super(message);  
    }  
}
```

// Custom Exception 3

```
class InvalidAmountException extends Exception {  
    InvalidAmountException(String message) {  
        super(message);  
    }  
}
```

// Wallet Class

```
class Wallet {  
    private String userName;
```

```
private String mobileNumber;  
private String upild;  
private double balance;
```

```
// Constructor
```

```
Wallet(String userName, String mobileNumber,  
        String upild, double balance) {  
    this.userName = userName;  
    this.mobileNumber = mobileNumber;  
    this.upild = upild;  
    this.balance = balance;  
}
```

```
// Add money
```

```
void addMoney(double amount) {  
    balance = balance + amount;  
    System.out.println("Money added: Rs." + amount);  
}
```

```
// Get balance
```

```
double getBalance() {  
    return balance;  
}
```

```
// Get UPI ID
```

```
String getUpild() {  
    return upild;  
}
```

```
// Deduct money
```

```
void deductMoney(double amount) {  
    balance = balance - amount;  
}
```

```
// Display wallet details
```

```

void displayWalletDetails() {
    System.out.println("\n===== WALLET DETAILS =====");
    System.out.println("User Name    : " + userName);
    System.out.println("Mobile Number : " + mobileNumber);
    System.out.println("UPI ID      : " + upild);
    System.out.println("Balance     : Rs." + balance);
}
}

```

// UPI Payment Class

```

class UPIPayment implements PaymentService {

```

```

    private Wallet wallet;

```

// Constructor

```

UPIPayment(Wallet wallet) {
    this.wallet = wallet;
}

```

// Payment method

@Override

```

public void pay(String upild, double amount) {

```

```

    try {

```

```

        // Validate UPI ID

```

```

        if (!upild.contains("@")) {

```

```

            throw new InvalidUPIException(

```

```

                "Invalid UPI ID!"

```

```

            );

```

```

        }

```

```

        // Validate amount

```

```

        if (amount <= 0) {

```

```

            throw new InvalidAmountException(

```

```

        "Amount must be greater than zero!"
    );
}

// Check balance
if (amount > wallet.getBalance()) {
    throw new InsufficientBalanceException(
        "Insufficient wallet balance!"
    );
}

// Deduct amount
wallet.deductMoney(amount);

System.out.println("Payment Successful!");
System.out.println("Paid Rs." + amount);
System.out.println("To UPI ID: " + upild);

} catch (InvalidUPIException e) {
    System.out.println("Transaction Failed: " + e.getMessage());

} catch (InvalidAmountException e) {
    System.out.println("Transaction Failed: " + e.getMessage());

} catch (InsufficientBalanceException e) {
    System.out.println("Transaction Failed: " + e.getMessage());

} finally {
    System.out.println("Transaction process completed.");
}
}

```

```

// Check balance
@Override

```

```
public void checkBalance() {  
    System.out.println("Available Balance: Rs."  
        + wallet.getBalance());  
}  
}
```

// Main Class

```
public class DigitalWallet {  
    public static void main(String[] args) {  
  
        // Create wallet  
        Wallet wallet = new Wallet(  
            "Deekshitha",  
            "9876543210",  
            "deekshitha@upi",  
            5000  
        );  
  
        // Create payment service  
        UPIPayment payment = new UPIPayment(wallet);  
  
        // Display initial wallet details  
        wallet.displayWalletDetails();  
  
        // Add money  
        System.out.println("\n===== ADD MONEY =====");  
        wallet.addMoney(2000);  
  
        // Check balance  
        System.out.println("\n===== BALANCE =====");  
        payment.checkBalance();  
    }  
}
```

// Successful payment

```
System.out.println("\n===== PAYMENT 1 =====");  
payment.pay("rahul@upi", 1500);
```

// Invalid UPI

```
System.out.println("\n===== PAYMENT 2 =====");  
payment.pay("rahulupi", 500);
```

// Invalid amount

```
System.out.println("\n===== PAYMENT 3 =====");  
payment.pay("rahul@upi", -100);
```

// Insufficient balance

```
System.out.println("\n===== PAYMENT 4 =====");  
payment.pay("rahul@upi", 10000);
```

// Final details

```
wallet.displayWalletDetails();
```

```
System.out.println("\n===== FINAL BALANCE =====");  
payment.checkBalance();
```

```
}
```

```
}
```

6. OUTPUT SCREENSHOT

```
PS C:\Users\Meghana\Desktop\javaprograms> java DigitalWallet
```

```
===== WALLET DETAILS =====
```

```
User Name      : Deekshitha
Mobile Number  : 9876543210
UPI ID         : deekshitha@upi
Balance        : Rs.5000.0
```

```
===== ADD MONEY =====
```

```
Money added: Rs.2000.0
```

```
===== BALANCE =====
```

```
Available Balance: Rs.7000.0
```

```
===== PAYMENT 1 =====
```

```
Payment Successful!
```

```
Paid Rs.1500.0
```

```
To UPI ID: rahul@upi
```

```
Transaction process completed.
```

```
===== PAYMENT 2 =====
```

```
Transaction Failed: Invalid UPI ID!
```

```
Transaction process completed.
```

```
===== PAYMENT 3 =====
```

```
Transaction Failed: Amount must be greater than zero!
```

```
Transaction process completed.
```

```
===== PAYMENT 4 =====
```

```
Transaction Failed: Insufficient wallet balance!
```

```
Transaction process completed.
```

```
===== WALLET DETAILS =====
```

```
User Name      : Deekshitha
Mobile Number  : 9876543210
UPI ID         : deekshitha@upi
Balance        : Rs.5500.0
```

```
===== FINAL BALANCE =====
```

```
Available Balance: Rs.5500.0
```

The above terminal-style output shows the result after compiling and executing the Java program.

7. RESULT

Thus, the Digital Payment System was successfully designed and implemented in Java. The program demonstrates interface implementation, encapsulation, constructors, object creation, String-based UPI validation, custom exceptions, try-catch-finally exception handling, wallet operations, UPI payment processing, balance checking, and transaction validation. The final wallet balance after adding Rs.5000 and paying Rs.1500 is Rs.3500.